

1. Functional Dependency (FD: $X \rightarrow Y$)

Schema Invariant

Formal Rule: In relation R , $X \rightarrow Y$ holds iff $\forall$ tuples $t_1, t_2 \in R$:
If $t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$ **MUST** equal $t_2[Y]$.

Determinant: X (LHS) $\xrightarrow{\text{[Decides]}}$ Dependent: Y (RHS)

- Instance vs Schema:** FD schema rule hota hai, data ke coincidence par depend nahi karta!
- Closure Property:** Har relation instance me integrity enforce karta hai.

2. FD Taxonomy & Classification

Core Theory

FD Type	Formal Condition	Example & Meaning
Trivial	$Y \subseteq X$	$\{\text{Roll, Name}\} \rightarrow \text{Roll}$ (Always true)
Non-Trivial	$X \cap Y = \emptyset$	$\text{Roll} \rightarrow \text{Name}$ (Real constraint)
Semi-Trivial	$Y \not\subseteq X \ \& \ X \cap Y \neq \emptyset$	$\{\text{Roll, Sub}\} \rightarrow \{\text{Sub, Marks}\}$
Full FD	$\forall X' \subset X, X' \not\rightarrow Y$	Y depends on FULL composite key (2NF)
Partial FD	$\exists X' \subset \text{Key}, X' \rightarrow Y$	Non-prime depends on part of key (Violates 2NF!)

3. Armstrong's Axioms (Sound & Complete Axiomatic Inference System)

F⁺ Generation

Primary Axiom 1: Reflexivity

If $Y \subseteq X \Rightarrow X \rightarrow Y$

Subset property: Har attribute set apne sub-parts ko trivially determine karta hai.

Primary Axiom 2: Augmentation

If $X \rightarrow Y \Rightarrow XZ \rightarrow YZ \ (\forall Z)$

Additive property: Dono taraf koi bhi attribute jod sakte hain.

Primary Axiom 3: Transitivity

If $X \rightarrow Y \ \& \ Y \rightarrow Z \Rightarrow X \rightarrow Z$

Chaining property: Dependency aage chain banakar transfer hoti hai.

Derived 4: Union / Additive

$X \rightarrow Y \ \& \ X \rightarrow Z \Rightarrow X \rightarrow YZ$

LHS same ho toh RHS combine ho sakta hai.

Derived 5: Decomposition

$X \rightarrow YZ \Rightarrow X \rightarrow Y \ \& \ X \rightarrow Z$

RHS split hota hai. LHS KABHI split nahi hota!

Derived 6: Pseudo-Transitivity

$X \rightarrow Y \ \& \ WY \rightarrow Z \Rightarrow WX \rightarrow Z$

Useful for proving implied dependencies.

Soundness: Derived FD hamesha physically valid hogi. | **Completeness:** Saari logically implied FDs (Closure F^+) nikal aayengi.

4. Attribute Closure Algorithm (X^+)Polynomial $O(|F| \cdot |R|)$

- Init:** Set $\text{closure} = X$.
- Loop:** Repeat for each $Y \rightarrow Z \in F$:
If $Y \subseteq \text{closure}$, then $\text{closure} = \text{closure} \cup Z$.
- Fixed-Point:** Loop tab tak chalao jab tak koi naya attribute add na ho.
Return closure as X^+ .

Key Applications:

- Check if $A \rightarrow B$ holds: Test if $B \subseteq A^+$.
- Super Key Test: Check if X^+ contains ALL attributes ($X^+ = R$).

5. Fast Candidate Key Discovery (3-Category Matrix)

AKTU Exam Cheat

Category	Attribute Position	Key Presence Rule
Category A (Essential)	NEVER on RHS	MUST be in EVERY Candidate Key!
Category B (Pivot)	Both on LHS & RHS	Test combinations with Category A.
Category C (Extraneous)	ONLY on RHS (Never LHS)	CANNOT be in ANY Candidate Key!

Prime Attribute: Member of ANY Candidate Key.

Non-Prime Attribute: NOT part of ANY Candidate Key.

6. Equivalence of FD Sets ($F \equiv G$)

F* = G* Test

Do sets F aur G tabhi equivalent hote hain jab dono ek doosre ko cover karein:

- Test 1 (G covers F): $\forall X \rightarrow Y \in F$, check if $Y \subseteq X^+_G$.
- Test 2 (F covers G): $\forall A \rightarrow B \in G$, check if $B \subseteq A^+_F$.

Both tests pass $\Rightarrow F \equiv G$. Polynomial time verification!

8. The 3 Database Anomalies

Root: Redundancy

- Insertion Anomaly:** Cannot insert an independent entity without a primary key (NULL not allowed).
- Deletion Anomaly:** Deleting one tuple inadvertently destroys unrelated secondary entity data.
- Update Anomaly:** Redundant copies cause data inconsistency if some rows fail to update.

7. Canonical Minimal Cover (Fc) 3-Step Method

Irreducible Set

- Singleton RHS:** Break $X \rightarrow ABC$ into $X \rightarrow A$, $X \rightarrow B$, $X \rightarrow C$.
- Remove Extraneous LHS:** In $AB \rightarrow C$, if $C \in (A)^+$ without B, drop B!
- Remove Redundant FDs:** For each $X \rightarrow Y$, test if $Y \in X^+_{(F - \{X \rightarrow Y\})}$. If YES, delete!

9. First Normal Form (1NF)

Atomic Values

- Rule:** All attribute domains must contain strictly **atomic (indivisible) values**.
- Eliminate Multi-valued attributes (lists/arrays in cells).
 - Decompose Composite attributes (Address $\rightarrow$ City, PIN).
 - Eliminate Repeating groups (numbered columns like Skill_1, Skill_2).

10. Core Normal Forms In-Depth (2NF, 3NF, BCNF)

Master Decision Matrix

Normal Form	Prerequisite	Formal Condition to Satisfy	Violation Signature	How to Decompose
2NF	1NF	No Partial Dependency: Every non-prime attribute must depend on the FULL candidate key.	Proper Subset of CK $\rightarrow$ Non-Prime	$R1(X', Y)$ with key X' , and $R2(R - Y)$.
3NF	2NF	No Transitive Dependency: For every non-trivial $X \rightarrow Y$: X is Super Key OR Y is Prime Attribute	Non-Key $\rightarrow$ Non-Prime (LHS not SK and RHS not Prime)	Bernstein Synthesis: $R_i = X \cup Y$ from Canonical Cover Fc. Add CK if needed.
BCNF	1NF	Strict Super Key: For every non-trivial $X \rightarrow Y$: X MUST BE A SUPER KEY OF R (Zero Prime exceptions!)	LHS is NOT a Super Key (Even if RHS is Prime!)	$R1(X \cup Y)$ and $R2(R - Y)$. Guaranteed lossless, but may lose FD preservation!

11. Special Theorem: Simple Candidate Keys

Speed Shortcut

Theorem: Agar kisi relation ke sabhi candidate keys **Single Attribute (Simple)** hain, toh wo relation **AUTOMATICALLY 2NF** me hota hai!

Proof: Single attribute ka koi non-empty proper subset nahi hota, isliye partial dependency mathematically impossible hai!

12. The Great 3NF vs BCNF Trade-Off

Industry Standard

- 3NF:** Always guarantees **Lossless Join AND Dependency Preservation**. Slight redundancy allowed.
- BCNF:** Eliminates ALL functional redundancy, but **MAY LOSE functional dependencies!**

13. Lossless Join (Non-Additive) Decomposition

Mandatory $R = R_1 \bowtie R_2$

Binary Decomposition Theorem: Split of R into R_1, R_2 is Lossless iff:

$(R_1 \cap R_2) \rightarrow R_1 \text{ OR } (R_1 \cap R_2) \rightarrow R_2$

Intuition: Common attribute set MUST be a **Super Key in at least one** table!

If this condition fails $\Rightarrow$ **Spurious Tuples** (fake records) appear upon natural join!

15. Multivalued Dependencies (MVD) & 4NF

Independent Multi-Facts

MVD Definition ($X \twoheadrightarrow Y$): Y values associated with X are independent of Z values. Causes $m \times n$ Cartesian explosion!

$t1[X]=t2[X] \Rightarrow \exists t3, t4$ with swapped Y and Z coordinates

4NF Condition: Relation in BCNF, and $\forall$ non-trivial $X \twoheadrightarrow Y$: **X MUST BE A SUPER KEY OF R.**

Fagin's Split: $R(X,Y,Z)$ decomposes into $R1(X,Y)$ and $R2(X,Z)$.

14. Dependency Preservation Property

$(F1 \cup \dots \cup Fk)^+ = F^+$

Decomposition is dependency preserving if all original constraints can be checked within individual tables without executing expensive cross-table joins.

- $Fi = \pi_{Ri}(F) = \{ X \rightarrow Y \in F^+ \mid XY \subseteq Ri \}$
- 3NF:** Always preserves all FDs ($\checkmark$).
- BCNF:** May not preserve some FDs ($\times$).

16. Join Dependencies (JD) & 5NF (PJNF)

Ultimate Projection Form

Join Dependency: $\bowtie[R1, R2, \dots Rn]$ holds if $R = \bowtie \pi_{Ri}(R)$. Models 3-way cyclic relationships.

5NF Condition: In 4NF, and for every non-trivial JD: **EVERY Ri MUST BE A SUPER KEY OF R.**

Inclusion Dependency (IND): $R[X] \subseteq S[Y]$.
Theoretical foundation of **Foreign Key & Referential Integrity!**

17. Universal AKTU 10-Mark Normalization Solving Algorithm

Guaranteed Full Marks

Step 1: Find All Candidate Keys

- Partition attributes: Cat A (Never RHS), Cat B (Both), Cat C (Never LHS).
- Compute $(Cat A)^+$. If equal to R $\Rightarrow$ Unique Candidate Key!
- If not, combine with Cat B singles/pairs. Identify Prime vs Non-Prime.

Step 2: Test 2NF (Partial Dependency)

- Is any Non-Prime attribute determined by a proper subset of CK?
- If YES $\Rightarrow$ Violates 2NF (Highest NF = 1NF). Decompose into $R1(X, Y)$ & $R2(R - Y)$.

Step 3: Test 3NF (Transitive Dependency)

- For every $X \rightarrow Y$: Is X a Super Key? OR Is Y a Prime Attribute?
- If neither holds $\Rightarrow$ Violates 3NF. Highest NF = 2NF. Decompose via Bernstein.

Step 4: Test BCNF (Strict Super Key)

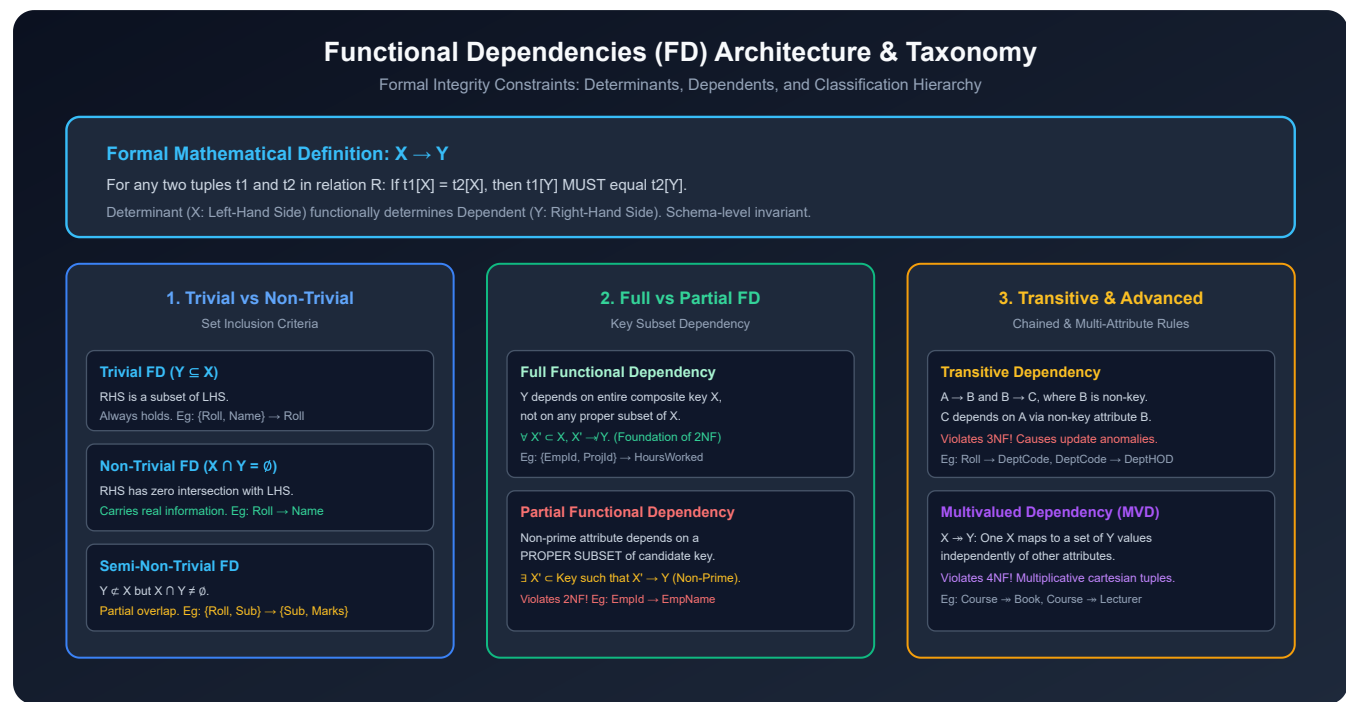
- For every $X \rightarrow Y$: Is X strictly a Super Key?
- If NO $\Rightarrow$ Highest NF = 3NF. Decompose R into $R1(X \cup Y)$ & $R2(R - Y)$.

18. Master Normal Form Inclusion & Anomaly Elimination Summary

Complete Hierarchy

Normal Form	Hierarchy Inclusion	Specific Anomaly Eliminated	Exam Test Rule
1NF	Base Relational	Non-atomic domains, multi-valued cells	All cell values atomic
2NF	$2NF \subset 1NF$	Partial dependency on composite keys	No Proper Subset of CK $\rightarrow$ Non-Prime
3NF	$3NF \subset 2NF$	Transitive dependency through non-keys	LHS Super Key OR RHS Prime Attribute
BCNF	$BCNF \subset 3NF$	Anomalies from overlapping candidate keys	LHS MUST be Super Key
4NF	$4NF \subset BCNF$	Cartesian explosion of independent facts	All MVD determinants are Super Keys
5NF (PJNF)	$5NF \subset 4NF$	N-way join spurious tuples & redundancy	All JD components are Super Keys

Module 01: Functional Dependencies & Types



> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Formal Definition of FD, Determinant vs Dependent, Schema Constraints vs Instance Data, Trivial, Non-Trivial, Semi-Trivial, Full vs Partial FDs.

1. Functional Dependency (FD) Core Concept & Philosophy

Database design mein Functional Dependency relation ke attributes ke beech ka ek **semantic integrity constraint** hota hai. Yeh decide karta hai ki ek table ke andar values kis tarah interrelated hain aur redundancy kahan paida ho rahi hai.

1.1 Formal Mathematical Definition

Ek relation schema $R(A_1, A_2, \dots, A_n)$ par, jisme $X \subseteq R$ aur $Y \subseteq R$ attributes ke sets hain:

$$X \rightarrow Y$$

$$\text{if } t_1[X] = t_2[X] \text{ then } t_1[Y] = t_2[Y]$$

Yeh Functional Dependency tabhi valid maani jaati hai jab relation R ke kisi bhi valid legal instance mein:

$$t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]$$

$$\text{forall } t_1, t_2 \in R, \quad \text{if } t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]$$

Bilingual Explanation: Agar do tuples (rows) t_1 aur t_2 ke paas attribute set X ki value exactly same hai, toh unke attribute set Y ki value bhi compulsory roop se **same honi chahiye!**

Determinant (X): Left-Hand Side (LHS) attribute jo decide karta hai.

Dependent (Y): Right-Hand Side (RHS) attribute jo decide hota hai.

,

+-----+ +-----+

LHS: X	—————>	RHS: Y
(Determinant)		(Dependent)

+-----+ +-----+

Eg: Roll_No Eg: Student_Name, Branch

,

Notice:

****Schema Constraint vs Instance Property:****

Functional Dependency table ke current snapshot ya instance ko dekh kar final nahi ki ja sakti! Yeh ****real-world business logic aur schema rules**** par depend karti hai.

Instance me do alag students ka naam same ho sakta hai ya coincidence se kuch rows match kar sakti hain, par FD schema level rule hota hai jo har bhavishya (future) ke instance par bhi hold karna chahiye.

2. Comprehensive Taxonomy of Functional Dependencies

Functional Dependencies ko unke attribute relationship aur mathematical properties ke basis par classify kiya jaata hai:

2.1 Trivial Functional Dependency

- **Condition:** $Y \subseteq X$ (RHS is a subset of LHS).
- **Explanation:** Jab right side ka attribute already left side ke attribute set ka part ho.
- **Validity:** Yeh har relation instance par **hamesha true** hoti hai, bina kisi data ko check kiye!
- **Examples:**

- $\{A, B\}$

$\rightarrow A$

- $\{Roll_No, Name\}$

$\rightarrow Roll_No$

- A

$\rightarrow A$

2.2 Non-Trivial Functional Dependency

- **Condition:** $X \cap Y = \emptyset$ (LHS and RHS have zero common attributes).
- **Explanation:** Jab right side ka koi bhi attribute left side me present nahi hota.
- **Significance:** Real-world database design me Non-Trivial FDs hi meaningful information aur constraints capture karti hain!
- **Examples:**

- $Roll_No$

$\rightarrow Student_Name$

- Emp_ID

$\rightarrow Salary$

- $\{Dept_ID, Project_ID\}$

$\rightarrow Budget$

2.3 Semi-Non-Trivial (Semi-Trivial) FD

- **Condition:** Y

$\subseteq X$ and $X \cap Y$

$\emptyset$.

- **Explanation:** Jab RHS ka kuch part LHS me common ho, par RHS poori tarah LHS ka subset na ho.

- **Example:**

- $\{A, B\}$

$\rightarrow \{B, C\}$

- By Decomposition Rule: $\{A, B\}$

$\rightarrow B$ (Trivial) and $\{A, B\}$

$\rightarrow C$ (Non-Trivial).

3. Full vs Partial Functional Dependencies (Gateway to 2NF)

Yeh distinction Normalization ke **Second Normal Form (2NF)** ka core pillar hai!

Parameter	Full Functional Dependency	Partial Functional Dependency
Formal Definition		

$\rightarrow Y$ holds, and X

forall $X' \subseteq X, X'$

ot

$\rightarrow Y \mid \exists X' \subseteq X$ such that X'

$\rightarrow Y \mid$

Simple Meaning	Y poore composite key par depend karta hai, kisi akele tukde par nahi.	Y composite key ke kisi ek chote hisse (proper subset) par depend kar jata hai.
Candidate Key Type	Composite Candidate Key required	Composite Candidate Key required
Impact on Database	Normalization ke liye desirable (2NF satisfied)	Redundancy paida karta hai (**Violates 2NF!**)
Real Example		

$\rightarrow Hours_Worked \mid \{Emp_ID, Project_ID\}$

$\rightarrow Emp_Name$ (Emp_ID

$\rightarrow Emp_Name$) |

4. Transitive Functional Dependency (Gateway to 3NF)

Definition: Jab ek non-key attribute kisi doosre non-key attribute par functionally depend karta hai through an indirect chain:

X

$Y \twoheadrightarrow Z$ and $Z \twoheadrightarrow X$

$Z \twoheadrightarrow X$

$X \twoheadrightarrow Z$

Where X is candidate key, but Y is NOT a super key and Z is a non-prime attribute.

Real-World Pitfall:

- Roll No

$\text{Department_Code} \twoheadrightarrow \text{Department_Head}$

- Department Code

$\text{Department_Head} \twoheadrightarrow \text{Roll_No}$

- Yahan Roll No

$\text{Department_Head} \twoheadrightarrow \text{Roll_No}$ indirectly transitive hai. Agar department head change hoga, toh har student ki row me update karna padega (Update Anomaly).

- Yeh dependency **Third Normal Form (3NF)** ko violate karti hai!

5. Architectural Diagram

6. Summary Comparison Table for Quick Revision

FD Type	Mathematical Form	Always Valid?	Violates Normal Form?	Real-Life Example
Trivial	$Y \subseteq X$	Yes (Trivially)	None	

$A \twoheadrightarrow B$

Non-Trivial	$X \cap Y = \emptyset$	Depends on schema	Core constraint
--------------------	------------------------	-------------------	-----------------

$\text{Emp_Salary} \twoheadrightarrow \text{Emp_Name}$

Semi-Trivial

$\emptyset \subseteq X$ and $X \cap Y$

$\emptyset$ | Depends on schema | Can be decomposed | $\{A, B\}$

$\{B, C\}$

Partial

$\rightarrow \text{ext}\{\text{Non-Prime}\} \mid \text{Depends on schema} \mid \textbf{Violates 2NF} \mid \Emp_ID

$\rightarrow \text{Emp_Name} \mid \text{in Project allocation} \mid$

Transitive

$\rightarrow X$

$\rightarrow Y \mid \text{Depends on schema} \mid \textbf{Violates 3NF} \mid \Student

$\rightarrow \text{Dept}$

$\rightarrow \text{HOD} \mid$

Multivalued	$X \twoheadrightarrow Y$	Independent multi-facts	Violates 4NF	$\text{Course} \twoheadrightarrow \text{Books}$
--------------------	--------------------------	-------------------------	---------------------	---

Module 02: Armstrong's Axioms & Inference Rules



$\rightarrow$ Roll_No\$.

2.2 Augmentation Rule (Additive Axiom)

Statement: Agar \$X

$\rightarrow$ Y\$, toh kisi bhi attribute set \$Z\$ ke liye \$XZ

$\rightarrow$ YZ\$ valid hoga.

Hindi: Dono taraf agar same attributes jod diye jayein, toh dependency intact rehti hai.

Example: Agar \$Roll_No

$\rightarrow$ Name\$, toh \$\{Roll_No, Semester\}

$\rightarrow$ \$\{Name, Semester\}\$.

2.3 Transitivity Rule (Chaining Axiom)

Statement: Agar \$X

$\rightarrow$ Y\$ aur \$Y

$\rightarrow$ Z\$, toh \$X

$\rightarrow$ Z\$ valid hoga.

Hindi: Dependency chain banakar aage propagate hoti hai.

Example: \$Roll_No

$\rightarrow$ Dept_ID\$ aur \$Dept_ID

$\rightarrow$ Dept_Name \$\implies\$ Roll_No

$\rightarrow$ Dept_Name\$.

3. Secondary / Derived Inference Rules (With Formal Proofs)

Primary axioms ko use karke hum practical calculations ke liye ye secondary rules derive karte hain:

3.1 Union / Additive Rule

Statement: Agar \$X

$\rightarrow$ Y\$ aur \$X

$\rightarrow$ Z\$, toh \$X

$\rightarrow$ YZ\$.

Formal Proof:

1. \$X

$\rightarrow$ Y\$ (Given)

2. X

$X \rightarrow X$ (Reflexivity) $\implies$ Augmenting step 1 by X : X

$X \rightarrow XY$.

3. X

$X \rightarrow Z$ (Given) $\implies$ Augmenting by Y : XY

$X \rightarrow YZ$.

4. By Transitivity (X

$X \rightarrow XY$ and X

$X \rightarrow YZ$): XY

$X \rightarrow YZ$. (Q.E.D.)

3.2 Decomposition / Projective Rule

Statement: Agar X

$X \rightarrow YZ$, toh X

$X \rightarrow Y$ aur X

$X \rightarrow Z$.

Formal Proof:

1. YZ

$X \rightarrow Y$ (Reflexivity, because $Y \subseteq YZ$).

2. X

$X \rightarrow YZ$ (Given).

3. By Transitivity: XY

$X \rightarrow Y$.

4. Similarly, YZ

$X \rightarrow Z$ (Reflexivity) $\implies$ By Transitivity: XY

$X \rightarrow Z$. (Q.E.D.)

Notice:

****CRITICAL AKTU EXAM TRAP:****

Right-Hand Side (RHS) attributes ko decompose kiya ja sakta hai (X

$X \rightarrow YZ \implies X$

$X \rightarrow Y, X$

$X \rightarrow Z$).

Par ****Left-Hand Side (LHS)** ko KABHI decompose nahi kiya ja sakta!**

$\$XY$

$\rightarrow Z \quad \mathbf{\{}$

$\ot \implies \quad X$

$\rightarrow Z \text{ ext } \{ \text{ or } \} Y$

$\rightarrow Z \$ \$$

Example: $\$(Date_of_Birth, Mother_Name\)$

$\rightarrow Person \$$, lekin akele $\$Date_of_Birth$

$\ot$

$\rightarrow Person \$!$

3.3 Pseudo-Transitivity Rule

- **Statement:** Agar $\$X$

$\rightarrow Y \$$ aur $\$WY$

$\rightarrow Z \$$, toh $\$WX$

$\rightarrow Z \$$.

- **Formal Proof:**

1. $\$X$

$\rightarrow Y \$$ (Given) $\implies$ Augment by $\$W \$$: $\$WX$

$\rightarrow WY \$$.

2. $\$WY$

$\rightarrow Z \$$ (Given).

3. By Transitivity ($\$WX$

$\rightarrow WY \$$ and $\$WY$

$\rightarrow Z \$$): **$\$WX$**

$\rightarrow Z \$$. (Q.E.D.)

3.4 Composition Rule

- **Statement:** Agar $\$X$

$\rightarrow Y \$$ aur $\$A$

$\rightarrow B \$$, toh $\$XA$

$\rightarrow YB \$$.

- **Formal Proof:**

1. $\$X$

$\rightarrow Y \implies XA$

$\rightarrow Y A \$$ (Augment by $A \$$).

2. A

$\rightarrow B \implies Y A$

$\rightarrow Y B \$$ (Augment by $Y \$$).

3. By Transitivity: $Y X A$

$\rightarrow Y B \$$.

4. Soundness and Completeness (Theoretical Guarantees)

Armstrong's Axiom system computer science ke sabse robust axiomatic frameworks me se ek hai kyunki yeh do essential properties satisfy karta hai:

1. Soundness (Sabhi derived FDs sahi hain):

- Agar koi FD X

$\rightarrow Y \$$ Armstrong axioms se derive hoti hai, toh wo relation $R \$$ ke har legal instance me 100% physically valid hogi. Koi bhi wrong/false dependency derive nahi ho sakti!

2. Completeness (Koi bhi valid FD chootegi nahi):

- Agar koi FD X

$\rightarrow Y \$$ relation schema ke rules ke hisaab se logically imply hoti hai, toh wo Armstrong axioms ke finite steps se zaroor derive ho sakti hai. Iska matlab closure $F^+ \$$ poori tarah calculate ho sakta hai!

5. Architectural Diagram

6. Solved AKTU Practice Derivation

Problem: Given $F = \{A$

$\rightarrow B, B$

$\rightarrow C, CD$

$\rightarrow E \setminus \$$. Prove that AD

$\rightarrow E \$$ holds using Armstrong's Axioms.

Solution Step-by-Step:

1. A

$\rightarrow B \$$ (Given)

2. B

$\rightarrow C \$$ (Given)

3. From (1) and (2), by **Transitivity Rule**: A

$\Rightarrow C$.

4. Take A

$\Rightarrow C$ and augment both sides with attribute D (**Augmentation Rule**):

AD

$\Rightarrow CD$

5. Given CD

$\Rightarrow E$.

6. From step 4 (AD

$\Rightarrow CD$) and step 5 (CD

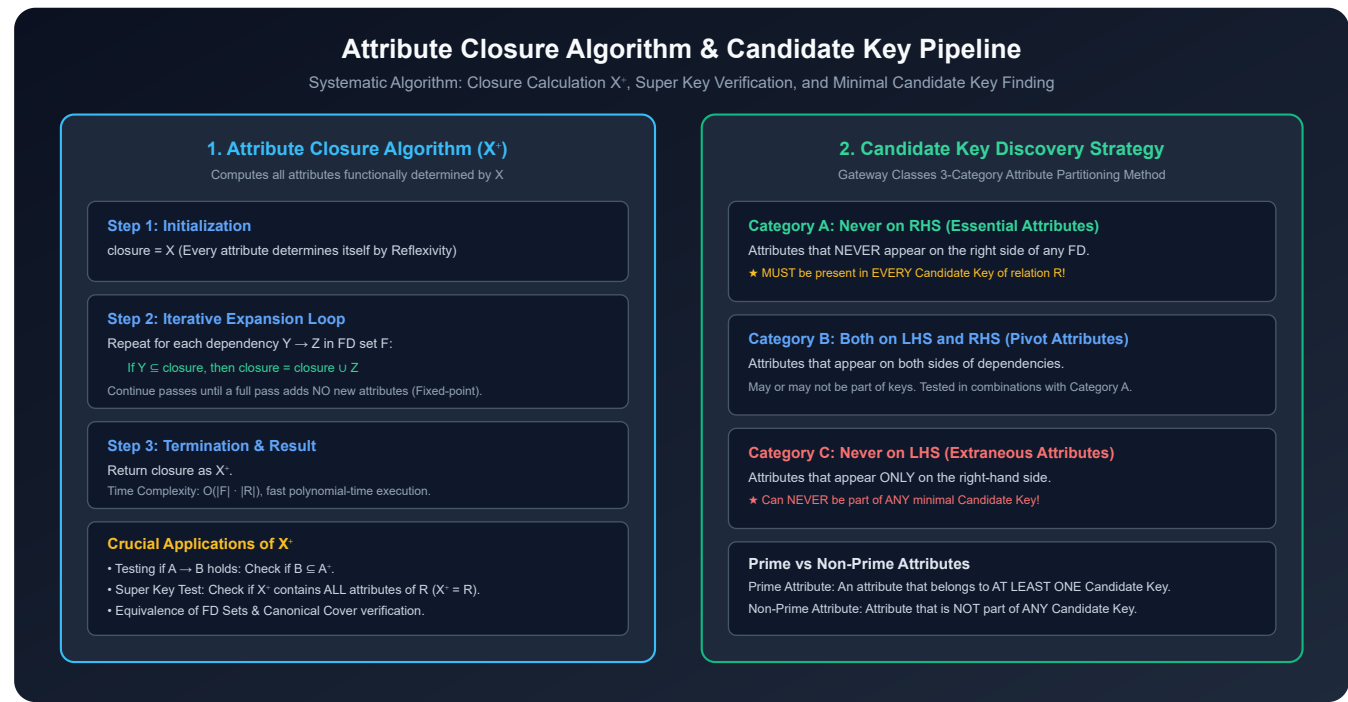
$\Rightarrow E$), by **Transitivity Rule**:

$\mathbf{AD}$

$\Rightarrow E$

Hence Proved!

Module 03: Attribute Closure & Candidate Key Algorithms



2. Core Applications of Attribute Closure in DBMS

1. Testing If a Functional Dependency Holds:

- Problem: Check if $X \twoheadrightarrow Y$
- Method: Compute X^+ . If $Y \subseteq X^+$, then **YES**, the dependency holds; otherwise NO.

2. Super Key Identification:

- If $X^+ = R$ (contains every attribute of the relation), then X is a **Super Key** of R .

3. Candidate Key Identification:

- If X is a Super Key AND no proper subset X' is a super key (minimality condition), then X is a **Candidate Key**!

3. Systematic Candidate Key Finding Algorithm (AKTU Gateway Method)

Jab ek relation me multiple attributes aur 5-6 FDs hoti hain, toh brute force har subset ka closure nikalna exponential (2^n) ho jaata hai. Gateway Classes aur standard database theory ka **3-Category Partitioning Method** candidate keys ko 30 seconds me solve kar deta hai:

Step 1: Divide all attributes of relation R into 3 categories:

Category A (Essential)	Category B (Pivot / Middle)	Category C (Extraneous)
Attributes that NEVER appear on the RHS of dependencies.	Attributes that appear on BOTH LHS & RHS	Attributes that appear ONLY on RHS (Never on LHS).
MUST be in EVERY CK!	May be combined with Cat A.	CANNOT be in ANY CK!

,

3.1 Step-by-Step Procedure

1. Find all attributes belonging to **Category A** (Never on RHS).
 2. Calculate closure of $(\text{Category A})^+$.
- **Case 1:** If $(\text{Category A})^+ = R$, then Category A is the **Unique Candidate Key**! You are finished!
 - **Case 2:** If $(\text{Category A})^+ \neq R$, then combine Category A with single attributes from **Category B**, then pairs, and check which combinations produce a closure equal to R .
3. Never include attributes from **Category C** in candidate key tests.

4. Prime vs Non-Prime Attributes

Yeh distinction 2NF aur 3NF testing ke liye mandatory hai:

- Prime Attribute:** Agar koi attribute relation ke **kisi bhi ek Candidate Key** ka hissa hai, toh use Prime Attribute kehte hain.
- Non-Prime Attribute:** Wo attributes jo relation ke **kisi bhi Candidate Key** me shaamil nahi hote.

Example: If Candidate Keys are $\{AB, BC\}$, then:

- Prime Attributes = $\{A, B, C\}$
- Non-Prime Attributes = All remaining attributes in R .

5. Architectural Diagram

6. Solved Gateway Classes AKTU Numerical Problems

Numerical 1 (From Gateway Slide Deck)

Question: Given relation $R(A, B, C, D, E, F)$ with Functional Dependencies:

$$F \rightarrow A$$

$$B \rightarrow B$$

$$C \rightarrow C$$

$$D \rightarrow D$$

$$E \rightarrow E$$

$$F \rightarrow F$$

Find all Candidate Keys of R , and classify Prime and Non-Prime attributes.

Solution:

1. Analyze Attributes:

- RHS contains: $\{B, C, D, E, F\}$.

- Attribute A never appears on the RHS $\implies$ Category A (Essential) = $\{A\}$.

2. Compute Closure of $\{A\}$:

$$A^+ = \{A\}$$

- Since A

$$B \in F \implies A^+ = \{A, B\}$$

- Since B

$$C \in F \implies A^+ = \{A, B, C\}$$

- Since C

$$D \in F \implies A^+ = \{A, B, C, D\}$$

- Since D

$$E \in F \implies A^+ = \{A, B, C, D, E\}$$

- Since E

$$F \in F \implies A^+ = \{A, B, C, D, E, F\} = R.$$

3. Conclusion:

- Since $A^+ = R$ and $\{A\}$ has no proper subset, **A is the only Candidate Key.**

- **Prime Attributes:** $\{A\}$

- **Non-Prime Attributes:** $\{B, C, D, E, F\}$

Numerical 2 (Multiple Candidate Keys)

Question: Given relation $R(A, B, C, D)$ with $F = \{AB$

$$\rightarrow CD, C$$

$$\rightarrow A, D$$

$$\rightarrow B\}. Find all Candidate Keys.$$

Solution:

1. Categorization:

- RHS has: $\{A, B, C, D\}$. Every attribute appears on RHS!

- Essential Category $A = \{\emptyset\}$.
- All attributes $\{A, B, C, D\}$ are in Category B.

2. Check pairs:

- $(AB)^+ = \{A, B, C, D\} = R \implies \mathbf{AB}$ is a Candidate Key.

- $(BC)^+$: Since C

$\rightarrow A, (BC)^+ \supseteq \{A, B, C\}$

$\rightarrow D \implies (BC)^+ = R \implies \mathbf{BC}$ is a Candidate Key.

- $(CD)^+$: Since C

$\rightarrow A$ and D

$\rightarrow B, (CD)^+ \supseteq \{A, B, C, D\} = R \implies \mathbf{CD}$ is a Candidate Key.

- $(AD)^+$: Since D

$\rightarrow B, (AD)^+ \supseteq \{A, B, D\}$

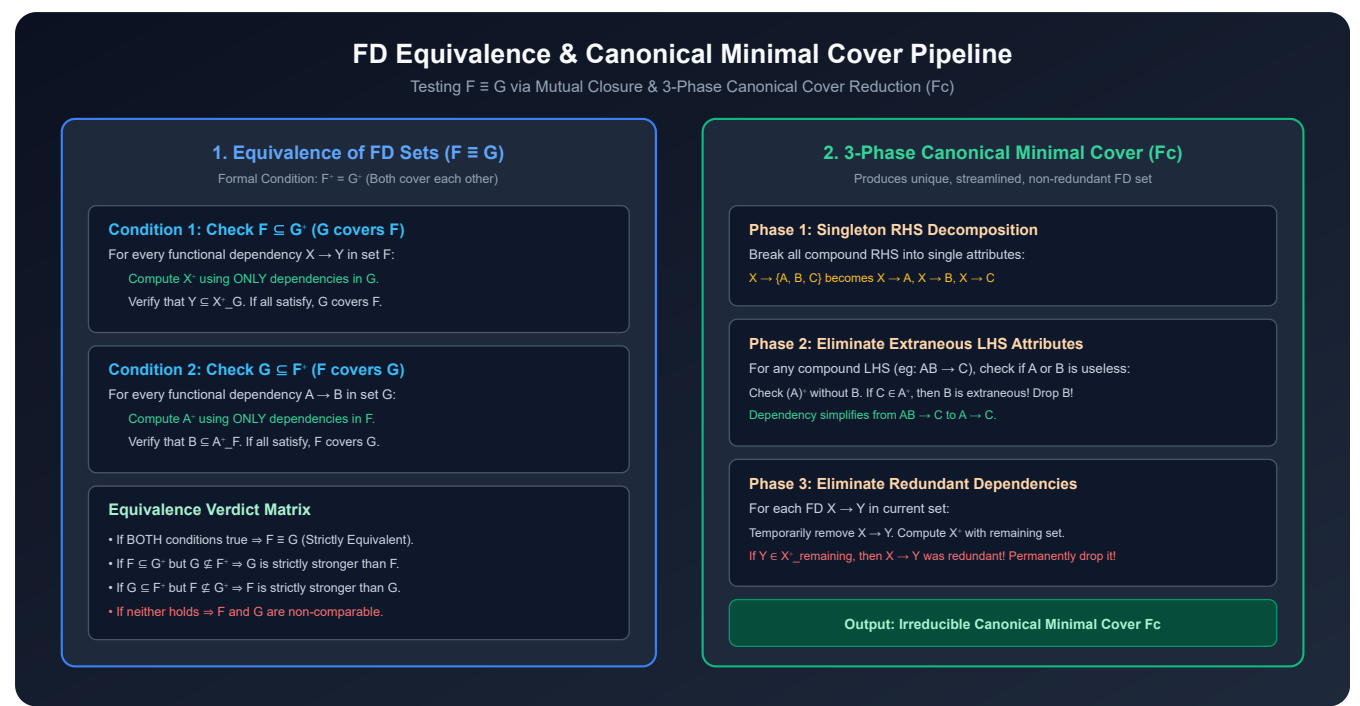
$\rightarrow C \implies (AD)^+ = R \implies \mathbf{AD}$ is a Candidate Key.

3. Total Candidate Keys: $\{AB, BC, CD, AD\}$.

4. **Prime Attributes:** $\{A, B, C, D\}$ (Every attribute is prime!).

5. **Non-Prime Attributes:** $\{\emptyset\}$ (None).

Module 04: Equivalence & Canonical Minimal Cover



$\rightarrow B \in G$:

- Compute the attribute closure of A using **ONLY the dependencies of F** (A^+_F).
- If $B \subseteq A^+_F$ holds for every FD in G , then F covers G .

3. Conclusion:

- If BOTH Test 1 and Test 2 pass $\implies \mathbf{F \equiv G}$.

2. Canonical Cover / Minimal Cover (F_c)

Database schema ko optimize karne ke liye hume aisa equivalent FD set chahiye jisme koi bhi extra (redundant) attribute ya rule na ho. Is minimal non-redundant set ko **Canonical Cover** (F_c) ya **Minimal Cover** kehte hain.

2.1 Formal Properties of a Canonical Cover

Ek set F_c canonical cover tabhi hota hai jab:

- 1. Singleton RHS:** Har FD ke right side par strictly **sirf ek attribute** ho (X

$\rightarrow A$).

- 2. No Extraneous LHS Attributes:** Left-hand side par koi faltu attribute na ho (e.g., agar A

$\rightarrow B$ se kaam chal raha hai toh AC

$\rightarrow B$ me C extraneous hai).

- 3. No Redundant Dependencies:** Koi aisi dependency na ho jisko hata dene par bhi closure me koi farq na pade.

3. The 3-Phase Canonical Cover Algorithm

,

Step 1: Right-Hand Side Decomposition

RHS ke sabhi multi-attributes ko singleton me tod do.

Eg: $X \rightarrow ABC$ becomes $\{X \rightarrow A, X \rightarrow B, X \rightarrow C\}$

Step 2: Extraneous Attribute Removal (from LHS)

Har composite LHS dependency (eg: $AB \rightarrow C$) ke liye check karo:

- Kya A ko hataya ja sakta hai? Calculate $(B)^+$. If $C \in (B)^+$, then A is extraneous!
- Kya B ko hataya ja sakta hai? Calculate $(A)^+$. If $C \in (A)^+$, then B is extraneous!
- Extraneous attribute ko permanently delete karo.

Step 3: Redundant Dependency Removal

Har remaining dependency $X \rightarrow Y$ ke liye:

- Us dependency ko temporarily set se bahar nikalo: $F' = F - \{X \rightarrow Y\}$.
- Remaining set F' ko use karke X^+ calculate karo.
- Agar $Y \in X^+_{F'}$, iska matlab $X \rightarrow Y$ baaki bachi FDs se already derive ho raha hai!
- Yeh redundant hai! Isko permanently delete karo!

,

4. Architectural Diagram

5. Gateway Classes Solved AKTU Examination Numericals

Numerical 1 (Full Canonical Cover Calculation)

Question: Given relation $R(A, B, C)$ and FD set:

$$F = \{ A$$

$$\rightarrow BC, \quad B$$

$$\rightarrow C, \quad A$$

$$\rightarrow B, \quad AB$$

$$\rightarrow C \}$$

Find the Canonical Cover (F_c).

Step 1: Singleton RHS Decomposition

$$A$$

$$\rightarrow BC \text{ splits into } A$$

$$\rightarrow B \text{ and } A$$

$$\rightarrow C.$$

Current set:

$$F_1 = \{ A$$

$$\rightarrow B, \quad A$$

$$\rightarrow C, \quad B$$

$$\rightarrow C, \quad A$$

$$\rightarrow B, \quad AB$$

$\rightarrow C \cup \{A\}$

Duplicate A

$\rightarrow B$ removed:

$F_1 = \{A\}$

$\rightarrow B, A$

$\rightarrow C, B$

$\rightarrow C, AB$

$\rightarrow C \cup \{A\}$

Step 2: Check Extraneous Attribute in AB

$\rightarrow C$

Check if B can be dropped from LHS:

- Compute closure of A without using AB

$\rightarrow C$:

- In $\{A\}$

$\rightarrow B, A$

$\rightarrow C, B$

$\rightarrow C \cup \{A\}, A^+ = \{A, B, C\}$.

- Since $C \in A^+$, A alone can derive C ! Therefore, **B is extraneous** in AB

$\rightarrow C$!

- Dependency AB

$\rightarrow C$ simplifies to A

$\rightarrow C$.

Since A

$\rightarrow C$ already exists in our set, duplicate is removed.

Set after Step 2:

$F_2 = \{A\}$

$\rightarrow B, A$

$\rightarrow C, B$

$\rightarrow C \cup \{A\}$

Step 3: Check Redundant Dependencies

1. Test A

$\Rightarrow B$:

- Remove it. Remaining: $\{A$

$\Rightarrow C, B$

$\Rightarrow C\}$.

- $A^+ = \{A, C\}$. Since B

$\in A^+, A$

$\Rightarrow B$ is **NOT redundant**. Keep it.

2. Test A

$\Rightarrow C$:

- Remove it. Remaining: $\{A$

$\Rightarrow B, B$

$\Rightarrow C\}$.

- $A^+ = \{A, B, C\}$ (using A

$\Rightarrow B$ and B

$\Rightarrow C$).

- Since $C \in A^+, A$

$\Rightarrow C$ is derived via transitivity!

- Therefore, A

$\Rightarrow C$ is REDUNDANT**! Remove it!

3. Test B

$\Rightarrow C$:

- Remove it. Remaining: $\{A$

$\Rightarrow B\}$.

- $B^+ = \{B\}$. Since C

$\in B^+, B$

$\Rightarrow C$ is **NOT redundant**. Keep it.

Final Answer:

$\mathbf{F}_c = \{ A$

$\Rightarrow B, \quad B$

$\rightarrow C \setminus \{x\}$

Module 05: Database Anomalies & 1NF

Database Anomalies & First Normal Form (1NF)

The Pitfalls of Data Redundancy: Insertion, Deletion, Update Anomalies & Atomic Domain Enforcement

Root Cause: Data Redundancy & Unnormalized Relation Flaws

1. Insertion Anomaly

Cannot insert independent entity without parent or primary key.

Pitfall: Cannot add new Department until at least one student enrolls (Primary Key cannot be NULL!).

2. Deletion Anomaly

Deleting one record inadvertently erases critical secondary facts.

Pitfall: Deleting the last student in Civil Engineering permanently destroys all Department and HOD data!

3. Update Anomaly

Redundant data stored across rows leads to severe inconsistency.

Pitfall: If Dept HOD changes, updating 500 student rows is required; missing 1 row creates split data.

First Normal Form (1NF) — Atomic Value Mandate

Eliminate Multivalued Values

A cell must contain only 1 value.

Violation: Phone: [981..., 942...]

Fix: Split into separate rows or a dedicated Student_Phone table.

Decompose Composite Values

Attributes must be indivisible.

Violation: Name: "Dr. Amit Sharma"

Fix: Decompose into First_Name, Middle_Name, Last_Name fields.

Eliminate Repeating Groups

Fixed numbered columns are invalid.

Violation: Skill_1, Skill_2, Skill_3...

Fix: Normalized relation with composite key: (Emp_ID, Skill).

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Relational Database Normalization Philosophy, Dr. E.F. Codd's Objectives, Insertion Anomaly, Deletion Anomaly, Modification/Update Anomaly, 1NF Atomic Value Rules, Repeating Groups Elimination.

1. Why Normalization? Core Philosophy & Objectives

Jab ek single table me multiple alag-alag entities ka data bina soche-samjhe combine kar diya jaata hai, toh database me **Data Redundancy** (unnecessary duplicate values) aa jaati hai.

Data Redundancy storage waste karti hai aur query performance degrade karti hai.
Sabse bada khatra: **Operational Anomalies** (Insert, Update, Delete ke waqt data corrupt hona).

Normalization ek systematic formal process hai jisme unnormalized relation ko progressively higher normal forms (1NF \$

ightarrow\$ 2NF \$

ightarrow\$ 3NF \$

ightarrow\$ BCNF) me decompose kiya jaata hai taaki:

- 1. Redundancy minimize ho sake.
- 2. Anomalies eliminate ho sakein.
- 3. Lossless join and integrity constraints preserve rahein.

2. The Three Classic Database Anomalies

Maan lijiye hamare paas ek combined table hai:

STUDENT_DEPT(Roll_No, Name, Branch, HOD, HOD_Cabin)

Roll_No	Name	Branch	HOD	HOD_Cabin
101	Aman	CSE	Dr. Sharma	Block-A, 101
102	Priya	CSE	Dr. Sharma	Block-A, 101
103	Rohan	ECE	Dr. Verma	Block-B, 204
104	Sneha	ME	Dr. Gupta	Block-C, 301

2.1 Insertion Anomaly

Definition: Kisi nayi information ko database me add na kar paana jab tak kisi unrelated entity ka data available na ho.

Example: College me ek naya department shuru hua: AI_DS, jiske HOD Dr. Kapoor hain. Lekin abhi tak kisi student ne admission nahi liya.

Problem: Kyunki Roll_No Primary Key hai (aur Primary Key NULL nahi ho sakti by Entity Integrity Constraint), hum naye department ka record table me **insert hi nahi kar sakte!**

2.2 Deletion Anomaly

Definition: Kisi ek entity ka data delete karte waqt kisi important secondary entity ka data accidentally permanent delete ho jaana.

Example: Mechanical Engineering (ME) department me sirf ek student 104 - Sneha padh rahi thi. Sneha ne college chhod diya aur uska record delete kiya gaya.

Problem: Sneha ki row delete hote hi ME branch, uske HOD Dr. Gupta, aur cabin number ka saara record database se **hamesha ke liye gayab ho gaya!**

2.3 Update / Modification Anomaly

Definition: Redundant data ko update karte waqt agar sabhi copies update na hon, toh data inconsistency create hona.

Example: CSE ke HOD Dr. Sharma retire ho gaye aur nayi HOD Dr. Anita ban gayi.

Problem: Agar CSE me 500 students hain, toh 500 rows me jakar HOD change karna padega. Agar server crash ya network issue ki wajah se 490 rows update hui aur 10 reh gayi, toh database contradictory answer dega!

3. First Normal Form (1NF)

3.1 Formal Definition of 1NF

A relation schema R is in **First Normal Form (1NF)** if and only if the domain of each attribute contains only **atomic (indivisible) values**, and each attribute value in a tuple is a **single value** from that domain.

3.2 1NF Violations and Solutions

1. Multi-Valued Attributes

Violation: Kisi ek cell ke andar multiple values pack karna:

Roll_No: 101 | Name: Aman | Mobile: [9899112233, 9811223344]

1NF Solution: Multiple values ko alag-alag rows me expand karo:

- Row 1: (101, Aman, 9899112233)

- Row 2: (101, Aman, 9811223344)

- Or create a separate related table: STUDENT_PHONE(Roll_No, Phone).

2. Composite Attributes

Violation: Aise attributes jo sub-parts me divide ho sakte hain unhe single column me rakhna:

Address: "Flat 402, Royal Apts, Sector 62, Noida, UP, 201301"

1NF Solution: Break into atomic sub-components: House_No, Street, Sector, City, State, Pincode.

3. Repeating Groups / Numbered Columns

Violation: Ek hi type ke data ke liye predefined numbered columns banana:

STUDENT(Roll_No, Skill_1, Skill_2, Skill_3)

1NF Solution: Dynamic child table create karo: STUDENT_SKILLS(Roll_No, Skill).

4. Architectural Diagram

5. Summary Checklist: Is Relation in 1NF?

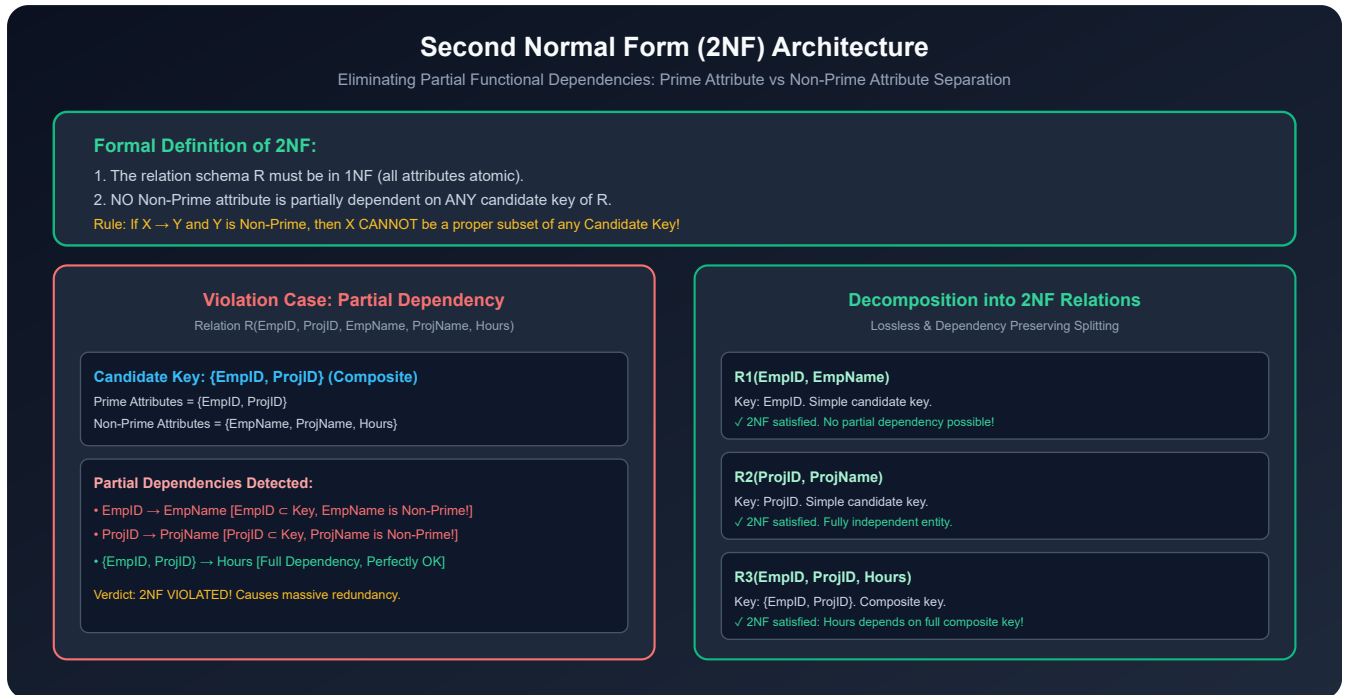
[x] Har column atomic values store karta hai? (No lists, no sets, no JSON arrays inside cells).

[x] Har row uniquely identifiable hai? (Primary Key identified).

[x] Columns ke names unique hain?

[x] Rows ka order meaningful nahi hai?

Module 06: Second Normal Form (2NF)



> **Course:** Database Management Systems (AKTU Code: **BCS501**)

Topic Focus: Second Normal Form Formal Criteria, Full Functional Dependency, Partial Functional Dependency Violation, Special Theorem for Simple Keys, 2NF Decomposition Algorithm with AKTU Solved Problem.

— — —

1. Second Normal Form (2NF) Formal Definition

Ek relation schema R **Second Normal Form (2NF)** me tab hota hai jab:

1. Wo relation already **1NF** me ho.
2. Us relation me **koi bhi Partial Functional Dependency** na ho.

1.1 Mathematical Formulation

\$\$

oral (X

$$\rightarrow Y \in F \quad \text{such that } Y \text{ is a Non-Prime Attribute}$$

$\mathbf{X}$ ext{ is NOT a proper subset of ANY Candidate Key of } R.

Prime Attribute: Jo kisi bhi ek candidate key ka member ho.

Non-Prime Attribute: Jo kisi bhi candidate key ka member na ho.

Partial Dependency: $\$ \text{ ext}\{\text{Proper Subset of Candidate Key}\}$

 $\rightarrow \text{ext}\{\text{Non-Prime Attribute}\}.$

— — —

2. The Powerful "Simple Candidate Key" Theorem (AKTU Shortcut)

Theorem: Agar kisi relation ke sabhi candidate keys **Simple (Single Attribute)** hain (yaani koi bhi candidate key composite nahi hai), toh wo relation **by default hamesha 2NF me hoga**!

Proof:

Partial dependency tabhi ho sakti hai jab candidate key ka koi proper non-empty subset ho. Agar Candidate Key $K = \{A\}$ hai, toh iska koi non-empty proper subset exist hi nahi karta! Isliye partial dependency create hona mathematically impossible hai!

3. Systematic 2NF Decomposition Algorithm

,

Input: Relation R with FD set F and Candidate Keys CK

Output: Set of relations in 2NF

1. Find all Candidate Keys of R.
2. Identify all Prime Attributes and Non-Prime Attributes.
3. Check every FD $X \rightarrow Y$ in F:

- If Y is Non-Prime AND X is a proper subset of some Candidate Key:

→ Partial Dependency Detected!

4. Decompose R:

- Create new relation $R_1(X, Y)$ with Primary Key X.
- Remove Y from original relation: $R_{\text{remaining}} = R - Y$.

5. Repeat until no partial dependencies remain.

,

4. Architectural Diagram

5. Gateway Classes Solved AKTU Examination Problem

Problem: Given relation $R(\text{Emp_ID}, \text{Project_ID}, \text{Emp_Name}, \text{Project_Name}, \text{Hours})$ with functional dependencies:

$F = \{ \text{Emp_ID} \rightarrow \text{Project_ID}, \text{Project_ID} \rightarrow \text{Hours} \}$

$\text{Emp_ID} \rightarrow \text{Hours}, \text{Emp_ID} \rightarrow \text{Project_ID}$

$\text{Emp_Name} \rightarrow \text{Project_ID}$

$\rightarrow \text{Project_Name} \rightarrow \text{Project_Name}$

1. Find Candidate Key:

- $\{ \text{Emp_ID}, \text{Project_ID} \}^+ = \{ \text{Emp_ID}, \text{Project_ID}, \text{Hours}, \text{Emp_Name}, \text{Project_Name} \} = R$.

- Candidate Key = $\{ \text{Emp_ID}, \text{Project_ID} \}$.

2. Classify Attributes:

- **Prime Attributes:** $\{ \text{Emp_ID}, \text{Project_ID} \}$

- **Non-Prime Attributes:** $\{ \text{Emp_Name}, \text{Project_Name}, \text{Hours} \}$

3. Test Each Functional Dependency for 2NF:

- $\{ \text{Emp_ID}, \text{Project_ID} \}$

$\rightarrow \text{Hours}$: Determinant is Full Candidate Key. **(Full FD - OK)**

- Emp_ID

$\rightarrow \text{Emp_Name}$: LHS $\text{Emp_ID} \subset \text{Candidate Key}$ and RHS Emp_Name is Non-Prime! **(Partial FD - VIOLATION!)**

- Project_ID

$\rightarrow \text{Project_Name}$: LHS $\text{Project_ID} \subset \text{Candidate Key}$ and RHS Project_Name is Non-Prime! **(Partial FD - VIOLATION!)**

4. Decomposition into 2NF:

- $R_1(\text{Emp_ID}, \text{Emp_Name})$ with Key = Emp_ID .

- $R_2(\text{Project_ID}, \text{Project_Name})$ with Key = Project_ID .

- $R_3(\text{Emp_ID}, \text{Project_ID}, \text{Hours})$ with Key = $\{ \text{Emp_ID}, \text{Project_ID} \}$.

5. **Conclusion:** All three relations R_1, R_2, R_3 are strictly in 2NF, lossless, and dependency preserving!

Module 07: Third Normal Form (3NF)

Third Normal Form (3NF) & Transitive Dependency

Universal 3NF Condition: Super Key LHS or Prime Attribute RHS & 3NF Synthesis

The Golden Rule of Third Normal Form (3NF):

Relation R is in 3NF iff for every non-trivial functional dependency $X \rightarrow Y$, at least ONE holds:

Condition 1: X is a **SUPER KEY** of R (X is **Key Determinant**)

OR Condition 2: Y is a **PRIME ATTRIBUTE** (Y is part of some **Candidate Key**)

Transitive Dependency Hazard

Chained Non-Key Determination: $\text{Key} \rightarrow X \rightarrow Y$

Example: Student(Roll, DeptCode, DeptHOD)

$\text{Roll} \rightarrow \text{DeptCode}$ and $\text{DeptCode} \rightarrow \text{DeptHOD}$

Neither DeptCode is Super Key NOR DeptHOD is Prime!

Why 3NF Prohibits This:

- If HOD changes, multiple student rows must update.
- If all students in a department leave, HOD info is lost!

Decomposition fix: S1(Roll, DeptCode) & S2(DeptCode, DeptHOD)

Lossless Join + Dependency Preservation guaranteed in 3NF!

3NF Synthesis Algorithm (Lossless + Preserving)

Bernstein's Proven Polynomial Procedure

Step 1: Compute Canonical Cover F_c

Simplify F into minimal canonical cover F_c .

Step 2: Create Sub-Relations

For each FD $X \rightarrow Y$ in F_c , make $R_i = X \cup Y$.

Step 3: Ensure Candidate Key Inclusion

If no R_i contains a Candidate Key of R, add $R_{key} = CK$.

Step 4: Eliminate Subsumed Relations

If $R_j \subseteq R_k$, discard R_j . Output final schema!

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Third Normal Form Criteria, Removal of Transitive Dependency, Universal 3NF Test Condition ($\$X\$$ is Super Key OR $\$Y\$$ is Prime Attribute), Bernstein's 3NF Synthesis Algorithm, Gateway Classes Solved Numericals.

1. Third Normal Form (3NF) Formal Definition

Ek relation schema $\$R\$$ **Third Normal Form (3NF)** me tab hota hai jab:

1. Wo relation already **2NF** me ho.
2. Us relation me **koi bhi Transitive Dependency na ho**.

1.1 The Universal 3NF Test Condition (AKTU Standard)

Relation $\$R\$$ ke har non-trivial functional dependency $\$X \rightarrow Y\$$ ke liye, neeche diye gaye do conditions me se **kam se kam ek condition** true honi chahiye:

$\$X\$$ is a Super Key of $\$R\$$

$\$Y\$$ is a Prime Attribute of $\$R\$$

1. $\$X\$$ is a **Super Key** of $\$R\$$ (Determinant super key hai)

$\$X\$$ is a **Super Key** of $\$R\$$

2. $\$Y\$$ is a **Prime Attribute** ($\$Y\$$ kisi bhi candidate key ka hissa hai)

[!TIP]

Agar kisi FD me LHS Super Key nahi hai, lekin uska RHS Prime Attribute hai, toh wo dependency ****3NF me fully ALLOWED hai****! Yeh 3NF ka sabse bada relaxer hai jo ise BCNF se alag banata hai!

2. Transitive Dependency Hazard

Transitive Dependency: Jab ek non-prime attribute kisi doosre non-prime attribute par depend karta hai through candidate key:

$XY \twoheadrightarrow Z$

$X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z$

$X \twoheadrightarrow Z$

$XY \twoheadrightarrow Z$

(Where XY is not a super key and Z is non-prime).

Practical Hazard: Redundancy, Update Anomaly, aur Deletion Anomaly paida hoti hai.

3. Bernstein's 3NF Synthesis Algorithm

Yeh algorithm guarantee karta hai ki relation 3NF me decompose ho jaye aur saath hi **Lossless Join** aur **Dependency Preservation** dono properties 100% preserve rahein:

,

Algorithm: 3NF_Synthesis(R, F)

1. Find Canonical Cover F_c of F .
2. For each FD $(X \rightarrow Y)$ in F_c :

Create a new relation schema: $R_i = X \cup Y$;

3. If none of the generated relations R_i contains a Candidate Key of R :

Create an additional relation R_{key} containing any candidate key of R ;

4. Eliminate redundant relations:

If $R_j \subseteq R_k$, remove R_j ;

5. Return the resulting set of relations.

,

4. Architectural Diagram

5. Gateway Classes Solved AKTU Examination Problem

Problem: Given relation $R(A, B, C, D)$ with FD set:

$$F = \{ AB$$

$$\rightarrow C, \text{quad } C$$

$$\rightarrow D, \text{quad } D$$

$$\rightarrow A \}$$

Find all Candidate Keys, determine the highest Normal Form of R , and decompose into 3NF if not already in 3NF.

Step 1: Find Candidate Keys

Attributes on RHS: $\{A, C, D\}$.

Attribute B never appears on RHS $\implies B$ must be in every key!

$(AB)^+ = \{A, B, C, D\} = R \implies \mathbf{AB}$ is a Candidate Key.

$(BC)^+ = \{B, C, D, A\} = R \implies \mathbf{BC}$ is a Candidate Key (via C

$$\rightarrow D$$

$$\rightarrow A).$$

$(BD)^+ = \{B, D, A, C\} = R \implies \mathbf{BD}$ is a Candidate Key (via D

$$\rightarrow A, \text{ then } AB$$

$$\rightarrow C).$$

Candidate Keys: $\{AB, BC, BD\}$.

Prime Attributes: $\{A, B, C, D\}$ (All attributes are Prime!).

Non-Prime Attributes: $\emptyset$ (None!).

Step 2: Determine Highest Normal Form

Let us test each FD in F :

1. AB

$$\rightarrow C:$$

- Is LHS AB a Super Key? **YES** (AB is a Candidate Key). $\implies$ Satisfies 3NF and BCNF.

2. C

$$\rightarrow D:$$

- Is LHS C a Super Key? **NO** ($C^+ = \{A, C, D\}$

$\neq R$).

- Is RHS D a Prime Attribute? **YES** (D is part of Candidate Key BD). $\implies$ **Satisfies 3NF!**

3. D

$$\rightarrow A:$$

- Is LHS D a Super Key? **NO** ($D^+ = \{A, D\}$

$\neq R$).

- Is RHS A a Prime Attribute? **YES** (A is part of Candidate Key AB). $A \rightarrow B$ implies **Satisfies 3NF!**

Conclusion:

Relation R is in **3NF** because for every FD where LHS is not a super key, RHS is a prime attribute!

Relation R is **NOT in BCNF** because in C

$A \rightarrow D$ and $D \rightarrow A$

$A \rightarrow C$, the determinants C and D are not super keys!

Highest Normal Form = 3NF.

Module 08: Boyce-Codd Normal Form (BCNF)

Boyce-Codd Normal Form (BCNF) Architecture

Strict Determinant Requirement: Every LHS Must Be a Super Key & 3NF vs BCNF Trade-Offs

Boyce-Codd Normal Form (BCNF / Strict 3.5NF):

Relation R is in BCNF iff for every non-trivial functional dependency $X \rightarrow Y$:

X MUST BE A SUPER KEY OF R. (No exceptions! Zero prime relaxers!)

3NF vs BCNF Strict Comparison

Mathematical Inclusions & Trade-Offs

Inclusion Hierarchy

$BCNF \subset 3NF \subset 2NF \subset 1NF$

Every BCNF is 3NF. But not every 3NF is BCNF!

Lossless Join Property

- 3NF Decomposition: Always Lossless (Non-additive).
- BCNF Decomposition: Always Lossless.

Dependency Preservation (The Critical Divergence)

- 3NF: ALWAYS preserves all functional dependencies.
- BCNF: MAY LOSE functional dependencies upon split!

Classic Gateway BCNF Problem

Relation R(Student, Subject, Teacher)

Given Dependencies:

1. (Student, Subject) $\rightarrow$ Teacher
2. Teacher $\rightarrow$ Subject (Each teacher teaches 1 subject)

Candidate Keys: {Student, Subject} and {Student, Teacher}

Why it is in 3NF but NOT in BCNF:

In Teacher $\rightarrow$ Subject:

- Teacher is NOT a Super Key (Fails BCNF!).
- Subject IS a Prime Attribute (Passes 3NF!).

Decomposing into R1(Teacher, Subject) & R2(Student, Teacher)

LOSES the dependency (Student, Subject) $\rightarrow$ Teacher!

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Boyce-Codd Normal Form (BCNF), Strict Super Key Determinant Condition, BCNF vs 3NF Deep Comparison, The Classic Database Trade-Off: Dependency Preservation vs Redundancy, Solved Gateway AKTU Numerical.

1. Motivation Behind BCNF

1974 me Raymond Boyce aur Edgar Codd ne observe kiya ki 3NF kuch specific conditions me redundancy aur anomalies ko poori tarah eliminate nahi kar pata.

Yeh situation tab aati hai jab ek table me **multiple overlapping Candidate Keys** hoti hain!
3NF ka relaxer rule ("Y\$ is a Prime Attribute") un dependencies ko allow kar deta hai jisme determinant Super Key nahi hota.

Is loophole ko close karne ke liye **Boyce-Codd Normal Form (BCNF)** introduce kiya gaya, jise aksar **3.5 Normal Form** bhi kaha jaata hai.

2. Formal Definition of BCNF

Ek relation schema \$R\$ **Boyce-Codd Normal Form (BCNF)** me tab hota hai jab:

1. Wo relation already **1NF** me ho.
2. Har non-trivial functional dependency \$X

ightarrow Y\$ ke liye:

\$X\$ **MUST BE A SUPER KEY OF \$R\$**

Yahan koi "OR" condition nahi hai! Determinant XY ko compulsory roop se Super Key hona hi padega!

3. Comprehensive Comparison: 3NF vs BCNF

Parameter	Third Normal Form (3NF)	Boyce-Codd Normal Form (BCNF)
-----------	-------------------------	-------------------------------

$Y \rightarrow X$ | XY is Super Key OR Y is Prime Attribute | XY MUST be Super Key |

Strictness	Relaxed (allows prime attribute on RHS)	Extremely strict (zero exceptions)
Lossless Join Decomposition	Always guaranteed (✓)	Always guaranteed (✓)
Dependency Preservation	Always guaranteed (✓)	NOT always possible (✗)
Storage Redundancy	Minimal (slight redundancy may remain)	Zero redundancy for functional dependencies
Practical Industry Choice	Most widely used in transactional DBs	Used when dependency loss is acceptable

4. Architectural Diagram

5. Classic Gateway Classes Solved Numerical: The Trade-Off Case

Problem: Given relation $R(\text{Student}, \text{Subject}, \text{Teacher})$ with functional dependencies:

$$F = \{ \text{Student} \rightarrow \text{Subject} \}$$

$$\text{Teacher} \rightarrow \text{Subject}$$

$$\text{Subject} \rightarrow \text{Teacher}$$

(Assumption: Har student ek subject ke liye ek teacher ko assign hota hai, aur har teacher sirf ek subject padhata hai).*

Step 1: Candidate Keys

$(\text{Student}, \text{Subject})^+ = \{ \text{Student}, \text{Subject}, \text{Teacher} \} = R \implies (\text{Student}, \text{Subject})$ is Candidate Key 1.

$(\text{Student}, \text{Teacher})^+ = \{ \text{Student}, \text{Teacher}, \text{Subject} \} = R \implies (\text{Student}, \text{Teacher})$ is Candidate Key 2.

Prime Attributes: $\{ \text{Student}, \text{Subject}, \text{Teacher} \}$ (All attributes are Prime!).

Step 2: 3NF Test

1. $(\text{Student}, \text{Subject})$

$$\text{Teacher} \rightarrow \text{Subject}$$

- LHS $(\text{Student}, \text{Subject})$ is a Candidate Key (Super Key). $\implies$ Satisfies 3NF.

2. Teacher

$\rightarrow \text{ext}\{\text{Subject}\}$:

- LHS $\text{ext}\{\text{Teacher}\}$ is NOT a Super Key.
- RHS $\text{ext}\{\text{Subject}\}$ IS a Prime Attribute (part of Candidate Key 1).
- $\implies$ **Satisfies 3NF!**

Verdict for 3NF: Relation R is in **3NF!**

Step 3: BCNF Test

Look at $\text{ext}\{\text{Teacher}\}$

$\rightarrow \text{ext}\{\text{Subject}\}$:

- Determinant $\text{ext}\{\text{Teacher}\}$ is **NOT a Super Key!**
- In BCNF, there is NO prime attribute relaxation!

Verdict for BCNF: Relation R **FAILS BCNF!**

Step 4: Decomposing into BCNF & The Great Loss

To bring into BCNF, decompose using the violating dependency $\text{ext}\{\text{Teacher}\}$

$\rightarrow \text{ext}\{\text{Subject}\}$:

$R_1(\text{Teacher}, \text{Subject})$ with Key = $\text{ext}\{\text{Teacher}\}$ (in BCNF).

$R_2(\text{Student}, \text{Teacher})$ with Key = $\{\text{ext}\{\text{Student}\}, \text{ext}\{\text{Teacher}\}\}$ (in BCNF).

Both sub-relations are in BCNF and the join is Lossless ($R_1 \cap R_2 = \{\text{ext}\{\text{Teacher}\}\}$, which is a key in R_1).

The Critical Flaw: Original dependency $\{\text{ext}\{\text{Student}\}, \text{ext}\{\text{Subject}\}\}$

$\rightarrow \text{ext}\{\text{Teacher}\}$ cannot be tested in either R_1 or R_2 alone!

Dependency Preservation is LOST!

Module 09: Decomposition Properties (Lossless & Preservation)

Relational Decomposition Properties

The Two Non-Negotiable Pillars: Lossless Join Property & Dependency Preservation

1. Lossless Join Decomposition (Non-Additive)

Mandatory Condition: $R = R_1 \bowtie R_2$ (Zero Spurious Tuples)

Mathematical Criteria for Binary Split:

A decomposition of R into R1 and R2 is Lossless iff:

$(R_1 \cap R_2) \rightarrow R_1 \text{ OR } (R_1 \cap R_2) \rightarrow R_2$

Intuition & Key Meaning:

The common attribute set $(R_1 \cap R_2)$ MUST be a **SUPER KEY** in at least one of the decomposed schemas!

If this condition fails, joining $R_1 \bowtie R_2$ produces extra fake, bogus records called "Spurious Tuples"!

Lossless Guarantees by Normal Form:

- 2NF Decomposition: Always Lossless (✓)
- 3NF Decomposition: Always Lossless (✓)
- BCNF Decomposition: Always Lossless (✓)
- 4NF & 5NF: Always Lossless (✓)

2. Dependency Preservation Property

Condition: $(F_1 \cup F_2 \cup \dots \cup F_k)^+ = F^+$

Formal Criteria:

For each decomposed relation Ri, project dependencies:

$F_i = \pi_{R_i}(F) = \{ X \rightarrow Y \in F^+ \mid XY \subseteq R_i \}$

Preservation holds if the union of all Fi implies every FD in original F.

Operational Benefit in Industry DBMS:

Integrity constraints can be validated on individual tables upon INSERT / UPDATE without executing heavy and expensive cross-table JOIN operations!

Preservation Status by Normal Form:

- 3NF Decomposition: ALWAYS Preserved (✓ guaranteed!)
- BCNF Decomposition: NOT always preserved (✗)

Some FDs may span across multiple tables in BCNF.

Trade-off: Pick 3NF for preservation, BCNF for zero redundancy.

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Relational Decomposition, Lossless (Non-Additive) Join Decomposition, Binary Decomposition Theorem, Spurious Tuples Hazard, Dependency Preservation Formal Verification, 3NF vs BCNF Synthesis Trade-Off.

1. Why Do We Decompose Relations?

Unnormalized tables me redundancy aur insertion/deletion/update anomalies hoti hain. Inhe hatane ke liye hum large relation R ko do ya do se zyada smaller relations $\{R_1, R_2, \dots, R_k\}$ me divide karte hain.

Lekin agar decomposition galat tareeqe se kiya jaye, toh do bade nuksaan hote hain:

1. Data Loss / Fake Data Creation (Lossy Join)

2. Constraint Enforcement Cost Explosion (Loss of Functional Dependencies)

Isliye relational database theory me decomposition ko valid maanne ke liye **do core properties** ka satisfy hona mandatory hai.

2. Property 1: Lossless Join Decomposition (Non-Additive Join)

2.1 Formal Definition

Ek relation R ka decomposition $\{R_1, R_2\}$ **Lossless Join Decomposition** tab kehlata hai jab unka Natural Join wapas exactly original relation R produce kare:

$$R_1 \bowtie R_2 = R$$

Agar $R \not\subseteq (R_1 \bowtie R_2)$ ho jaye, yaani join karne par extra rows (fake data) generate ho jayein, toh use **Lossy (Additive) Decomposition** kehte hain.

Un extra fake rows ko **Spurious Tuples** kaha jaata hai!

2.2 Binary Decomposition Theorem (Universal AKTU Test)

Do sub-relations R_1 aur R_2 ke binary decomposition ke lossless hone ki **necessary and sufficient condition** yeh hai:

$$\mathbf{R_1 \cap R_2}$$

$$\rightarrow R_1 \text{ OR } R_2$$

$$\rightarrow R_2$$

Hindi Intuition: Dono tables ke common attributes $(R_1 \cap R_2)$ ko kam se kam kisi **ek table ka Super Key** hona hi chahiye!

3. Spurious Tuples Hazard (Illustrated Example)

Maan lijiye relation hai EMP_PROJ(EmpId, ProjId, Location):

Suppose isko galat attributes par decompose kiya gaya:

$$R_1(\text{EmpId}, \text{Location})$$

$$R_2(\text{ProjId}, \text{Location})$$

Common attribute is Location. Lekin Location na toh R_1 ka key hai, na R_2 ka!

Jab hum $R_1 \bowtie R_2$ karenge on Location, toh ek hi location par kaam karne wale saare employees saare projects ke saath cross-join ho jayenge, jisse **fake assignment records (Spurious Tuples)** ban jayenge!

User ko lagega ki Employee X Project Y par kaam kar raha hai jabki wo actually nahi kar raha tha!

4. Property 2: Dependency Preservation

4.1 Formal Definition

Decomposition $\{R_1, R_2, \dots, R_k\}$ **Dependency Preserving** tab hota hai jab original set F ki saari functional dependencies decomposed tables me individually test ki ja sakein:

$$(F_1 \cup F_2 \cup \dots \cup F_k)^+ = F^+$$

Where F_i represents the projection of F onto R_i :

$$F_i = \pi_{R_i}(F) = \{ X$$

$$\rightarrow Y \mid XY \subseteq R_i \}$$

4.2 Why is Dependency Preservation Crucial in Industry?

Agar dependency preserve hoti hai, toh jab bhi koi user table me naya record INSERT ya UPDATE karta hai, database engine sirf usi single table par constraint check karta hai ($O(1)$ fast check).

Agar dependency preserve nahi hui, toh har chote insert par database ko multiple tables ka join perform karna padega jo system ko severely slow kar dega!

5. Architectural Diagram

6. Gateway Classes Solved AKTU Examination Problem

Problem: Relation $R(A, B, C, D)$ has FD set:

$$F = \{ A$$

$$\rightarrow B, \quad B$$

$$\rightarrow C, \quad C$$

$$\rightarrow D, \quad D$$

$$\rightarrow A \}$$

Relation is decomposed into:

$$R_1(A, B), R_2(B, C), \text{ and } R_3(C, D).$$

Check whether this decomposition is:

1. Lossless Join
2. Dependency Preserving

Step 1: Lossless Join Check

We can combine sequentially:

1. Join $R_1(A, B)$ and $R_2(B, C)$:

$$\text{- Common attribute} = R_1 \cap R_2 = \{B\}.$$

$$\text{- In } R_2, B$$

$$\rightarrow C \text{ holds, so } B^+ = \{B, C, D, A\} \supseteq R_2.$$

$$\text{- Thus, } B \text{ is a Super Key in } R_2!$$

$$\text{- Therefore, the join of } R_1 \text{ and } R_2 \text{ is } \mathbf{Lossless}, \text{ forming } R_{12}(A, B, C).$$

2. Join $R_{12}(A, B, C)$ and $R_3(C, D)$:

$$\text{- Common attribute} = R_{12} \cap R_3 = \{C\}.$$

$$\text{- In } R_3, C$$

$$\rightarrow D \text{ holds, so } C \text{ is a Super Key in } R_3!$$

- Therefore, the join of R_1 and R_3 is **Lossless**, forming $R(A, B, C, D)$.

Verdict 1: Decomposition is **Lossless Join!**

Step 2: Dependency Preservation Check

In $R_1(A, B)$: $A \rightarrow B$

$A \rightarrow B$ is preserved.

In $R_2(B, C)$: $B \rightarrow C$

$B \rightarrow C$ is preserved.

In $R_3(C, D)$: $C \rightarrow D$

$C \rightarrow D$ is preserved.

The only remaining FD in R is $D \rightarrow A$

$D \rightarrow A$:

- From R_3 , D

$D \rightarrow C$ is implied? $D \rightarrow C$

$D \rightarrow A$

$D \rightarrow B$

$D \rightarrow C$ implies $D \rightarrow A$ via transitivity:

$(F_1 \cup F_2 \cup F_3)$ has A

$D \rightarrow B$

$D \rightarrow C$

$D \rightarrow D$.

- Wait, does $D \rightarrow A$

$D \rightarrow B$

$D \rightarrow C$

$D \rightarrow D$ derive D

$D \rightarrow A$?

- $D^+_{\{A, B, C\}} = D^+$. It does NOT derive A !

Verdict 2: Dependency $D \rightarrow A$

$D \rightarrow A$ is **LOST!** Decomposition is **NOT Dependency Preserving**.

Module 10: Multivalued Dependencies & 4NF

Multivalued Dependencies (MVD) & Fourth Normal Form (4NF)

Independent Multi-Valued Facts, Cartesian Tuple Multiplication & 4NF Decomposition

Formal Definition of Multivalued Dependency: $X \twoheadrightarrow Y$

In relation schema $R(X, Y, Z)$, $X \twoheadrightarrow Y$ holds iff for any two tuples t_1 and t_2 with $t_1[X] = t_2[X]$, there MUST also exist tuples t_3 and t_4 in R such that:

$t_3 = (t_1[X], t_1[Y], t_2[Z])$ and $t_4 = (t_1[X], t_2[Y], t_1[Z])$

The MVD Cartesian Explosion Hazard

Table: Course_Info(Course, Lecturer, TextBook)

Independent Facts Combine Multiplicatively:

- Course 'DBMS' has 2 Lecturers: {Dr. A, Dr. B}
- Course 'DBMS' has 2 Textbooks: {Korth, Navathe}

Requires $2 \times 2 = 4$ tuples in a single unnormalized table!

Why BCNF Cannot Catch This:

There are NO functional dependencies! Every attribute is part of the primary key: {Course, Lecturer, TextBook}.

Table is in BCNF, yet suffers from massive redundancy and update anomalies! We need 4NF!

Fourth Normal Form (4NF) Solution

Isolating Independent Multi-Valued Facts

Formal 4NF Condition:

Relation R is in 4NF iff it is in BCNF and for every non-trivial MVD $X \twoheadrightarrow Y$:

X MUST BE A SUPER KEY OF R .

4NF Decomposition Method:

Decompose $R(X, Y, Z)$ with $X \twoheadrightarrow Y$ into two schemas:

- $R_1(X, Y)$ with Key = $\{X, Y\}$
- $R_2(X, Z)$ with Key = $\{X, Z\}$

Decomposed relations: $2 + 2 = 4$ rows instead of cross product!

Lossless Join guaranteed by Fagin's Theorem!

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Multivalued Dependencies (MVD: $\$X \twoheadrightarrow Y\$$), Limitations of Functional Dependencies, Independent Multivalued Facts, 4NF Definition, Fagin's Theorem, Solved AKTU Example.

1. Why Functional Dependencies Fall Short

Functional Dependencies ($\$X \rightarrow Y\$$)

only 1-to-1 ya many-to-1 associations ko model kar sakti hain.

Lekin real-world me aksar ek entity ke multiple independent attributes hote hain (1-to-many ya many-to-many). Jab do ya do se zyada independent multi-valued attributes ko ek hi relation me rakh diya jaata hai, toh **Cartesian Product Explosion** hota hai!

Is redundancy ko Functional Dependencies detect hi nahi kar paati kyunki koi attribute doosre ko uniquely determine nahi karta. Table ka poora attribute combination milkar Primary Key ban jaata hai, jisse relation BCNF satisfy kar leta hai par tab bhi corrupt aur redundant hota hai!

2. Multivalued Dependency (MVD: $\$X \twoheadrightarrow Y\$$)

2.1 Formal Definition

Relation $R(X, Y, Z)$ me jisme $Z = R - (X \cup Y)$, dependency:

$\$X \twoheadrightarrow Y\$$

Hold karti hai agar aur sirf agar kisi bhi relation instance me do tuples t_1 aur t_2 ke liye jahan $t_1[X] = t_2[X]$, relation me do aur tuples t_3 aur t_4 ka hona **anivarya (mandatory)** ho:

$$t_3 = (t_1[X], \text{quad } t_1[Y], \text{quad } t_2[Z])$$

$$t_4 = (t_1[X], \text{quad } t_2[Y], \text{quad } t_1[Z])$$

2.2 Hindi Intuition

Iska seedha matlab yeh hai ki attribute Y ki values jo X se judi hain, wo Z attribute ki values se **completely independent** hain!

Agar X ke paas m values of Y hain aur n values of Z hain, toh table me compulsory roop se m imes n tuples hone padenge!

3. Properties of Multivalued Dependencies

1. Complementation Rule:

- Agar $X \twoheadrightarrow Y$, toh $X \twoheadrightarrow (R - X - Y)$ bhi hamesha hold karega!
- (MVDs hamesha pairs me aati hain!).

2. Trivial MVD:

- Ek MVD $X \twoheadrightarrow Y$ trivial hoti hai agar:
- $Y \subseteq X$, ya fir
- $X \cup Y = R$.

3. FD implies MVD:

- Agar $X \rightarrow Y$, toh $X \twoheadrightarrow Y$ automatically true hota hai (MVD is a generalization of FD).

4. Fourth Normal Form (4NF) Formal Definition

A relation schema R is in **Fourth Normal Form (4NF)** with respect to a set of dependencies D (FDs and MVDs) if and only if it is in **BCNF**, and for every non-trivial Multivalued Dependency $X \twoheadrightarrow Y$ in D :

$$\mathbf{X \text{ is a SUPER KEY of } R}$$

4.1 Fagin's Theorem (4NF Decomposition)

Agar relation $R(X, Y, Z)$ me MVD $X \twoheadrightarrow Y$ hold karti hai aur X super key nahi hai, toh R ko do relations me decompose kiya ja sakta hai:

$$R_1(X, Y) \text{ and } R_2(X, Z)$$

Yeh decomposition hamesha **Lossless Join** hota hai!

5. Architectural Diagram

6. Gateway Classes Solved AKTU Examination Problem

Problem: Relation STUDENT_INFO(Roll_No, Hobby, Language)

Assume ek student ke multiple hobbies aur multiple known languages ho sakte hain, aur hobby aur language ke beech koi direct relation nahi hai.

1. Identify Dependencies:

- $\text{Roll_No} \twoheadrightarrow \text{Hobby}$
- $\text{Roll_No} \twoheadrightarrow \text{Language}$

2. Current Keys & Normal Form:

- No attribute determines another. Candidate Key = $\{\text{Roll_No}, \text{Hobby}, \text{Language}\}$.
- All FDs trivial $\implies$ Relation is in **BCNF**!

3. Check 4NF:

- In $\text{Roll_No} \twoheadrightarrow \text{Hobby}$, the determinant Roll_No is **NOT a Super Key**!
- Therefore, relation **FAILS 4NF**!

4. 4NF Decomposition:

- $R_1(\text{Roll_No}, \text{Hobby})$
- $R_2(\text{Roll_No}, \text{Language})$

5. **Result:** Both R_1 and R_2 are in 4NF, redundancy is eliminated, and join is lossless!

Module 11: Join Dependencies (5NF) & Inclusion Rules

Join Dependencies (JD), 5NF & Inclusion Dependencies

Project-Join Normal Form (PJNF), N-Way Lossless Join & Master Normal Form Hierarchy

1. Join Dependency (JD) & 5NF (PJNF)

N-way lossless decomposition beyond binary pairs

Join Dependency Definition: $\bowtie[R_1, R_2, \dots, R_n]$
Relation R satisfies JD $\bowtie[R_1, R_2, \dots, R_n]$ iff:
 $R = \pi_{R_1}(R) \bowtie \pi_{R_2}(R) \bowtie \dots \bowtie \pi_{R_n}(R)$

Fifth Normal Form (5NF / PJNF) Criteria:
Relation R is in 5NF iff it is in 4NF and for every non-trivial Join Dependency $\bowtie[R_1, R_2, \dots, R_n]$:
EVERY R_i MUST BE A SUPER KEY OF R.
If this holds, table cannot be losslessly decomposed any further!

Inclusion Dependencies (IND): $R[X] \subseteq S[Y]$
Subset constraints between relations.
Every value appearing in column X of R must also appear in column Y of S.
★ Direct formal basis of FOREIGN KEY and Referential Integrity!

2. Master Normal Form Hierarchy

Progressive Constraint Nesting: $5NF \subset 4NF \subset BCNF \subset 3NF \subset 2NF \subset 1NF$

1NF: Atomic Domains
Eliminates multi-valued attributes and composite attributes.

2NF: No Partial Dependencies
Non-prime attributes must depend on the FULL candidate key.

3NF: No Transitive Dependencies
For $X \rightarrow Y$: X is Super Key OR Y is Prime. Lossless + Preserved!

BCNF: Strict Super Key Determinant
For all $X \rightarrow Y$: X MUST be Super Key. May lose dependency preservation.

4NF: No Multi-Valued Dependencies
For all $X \twoheadrightarrow Y$: X is Super Key. Eliminates cartesian explosion.

5NF: No Join Dependencies (PJNF)
All non-trivial JDs implied by candidate keys. Zero join redundancy.

> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Join Dependencies $\bowtie[R_1, \dots, R_n]$, Fifth Normal Form (5NF / Project-Join Normal Form PJNF), Cyclic Constraint Dependencies, Inclusion Dependencies (IND) and Foreign Key Relationship, Master Normal Form Summary.

1. Limitations of 4NF: The 3-Way Cyclic Join

4NF sirf do independent multi-valued attributes ke binary splits ko handle karta hai (R

$\rightarrow R_1, R_2$).

Lekin database design me aisi cyclic relationships ho sakti hain jinhe do tables me split karne par **lossless join nahi milta**, lekin **teen ya usse zyada tables me split karne par lossless join mil jaata hai!**

Is generalisation ko capture karne ke liye **Join Dependency (JD)** define ki jaati hai.

2. Join Dependency (JD) Formal Definition

Ek relation schema R par **Join Dependency** $\bowtie[R_1, R_2, \dots, R_n]$ tab hold karti hai jab relation R apne projections ke natural join ke strictly equal ho:

$$R = \pi_{R_1}(R) \bowtie \pi_{R_2}(R) \bowtie \dots \bowtie \pi_{R_n}(R)$$

Where each $R_i \subseteq R$ and $\bigcup_{i=1}^n R_i = R$.

Trivial Join Dependency: Agar kisi ek $R_i = R$ ho jaye, toh JD trivial hoti hai.

3. Fifth Normal Form (5NF / Project-Join Normal Form - PJNF)

3.1 Formal Definition

Ek relation schema R **Fifth Normal Form (5NF)** me tab hota hai jab:

1. Wo relation already **4NF** me ho.
2. Har non-trivial Join Dependency $\bowtie[R_1, R_2, \dots, R_n]$ ke liye:

$\mathbf{\{ \text{EVERY } R_i \text{ IS A SUPER KEY OF } R \}}$

3.2 Significance

5NF relational database theory ki ultimate normal form maani jaati hai (with respect to projection and join operators).

Agar koi table 5NF me hai, toh use aage kisi bhi number of tables me bina spurious tuples ke split nahi kiya ja sakta jab tak un tables ka key original relation ka super key na ho!

4. Inclusion Dependencies (IND)

4.1 Formal Definition

Given two relation schemas R and S , attribute sequences $X \subseteq R$ and $Y \subseteq S$ (with matching domain and length):

$\mathbf{R[X] \subseteq S[Y]}$

Iska matlab hai ki relation R ke kisi bhi tuple me attribute X ki jo values aayengi, unka relation S ke attribute Y me pehle se present hona mandatory hai!

4.2 Connection to Foreign Key Constraints

Inclusion dependency relational database me **Foreign Key (Referential Integrity)** ka formal theoretical foundation hai!

SQL syntax:

```
`sql
```

```
FOREIGN KEY (Dept_ID) REFERENCES DEPARTMENT(Dept_ID);
```

,

Is directly the Inclusion Dependency: $\{ \text{EMPLOYEE}[\text{Dept_ID}] \subseteq \text{DEPARTMENT}[\text{Dept_ID}] \}$.

5. Architectural Diagram

6. Master Summary of All Normal Forms (AKTU Quick Reference Table)

Normal Form	Core Requirement	Violation Removed	Lossless Join?	Dependency Preserving?
1NF	Atomic attribute values only	Multi-valued & composite attributes	\$\checkmark\$	\$\checkmark\$
2NF	In 1NF + No Partial Dependencies	Non-prime depending on proper subset of key	\$\checkmark\$	\$\checkmark\$
3NF				

ightarrow Y\$: \$X\$ is Super Key **OR** \$Y\$ is Prime | Transitive dependencies (Non-prime \$

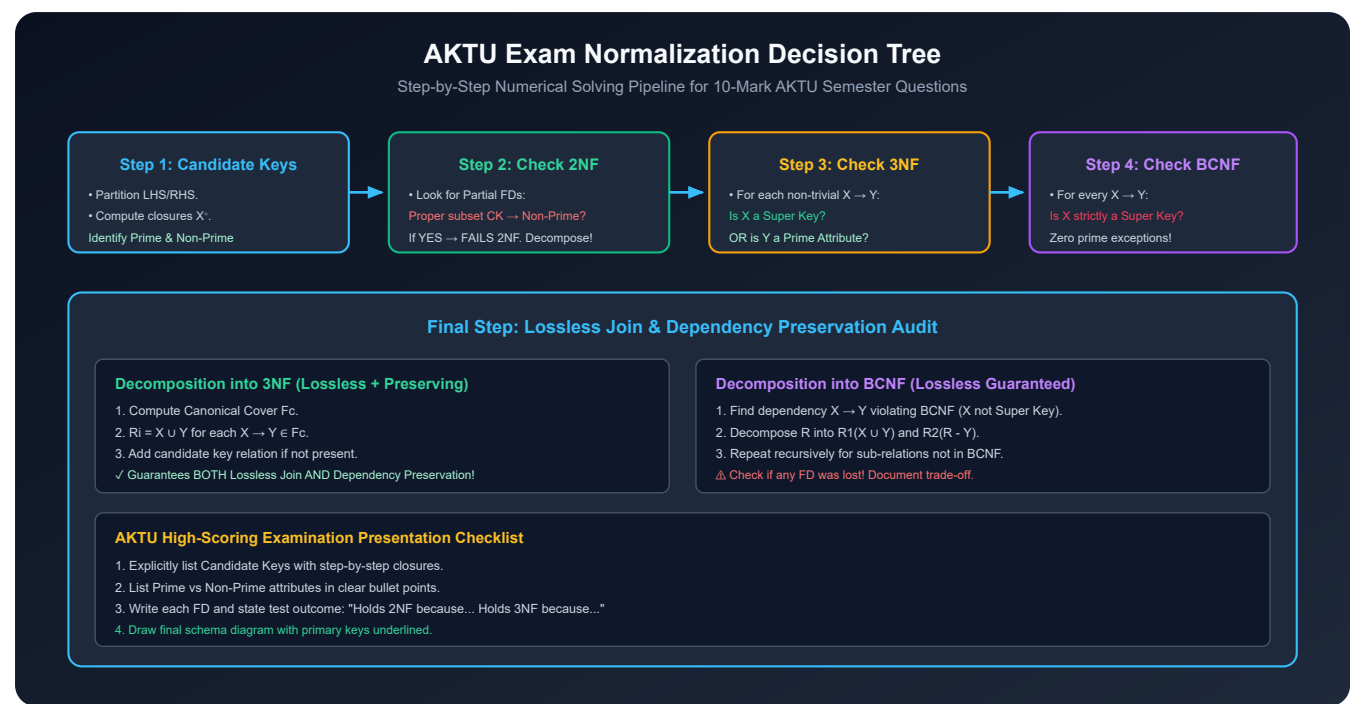
ightarrow\$ Non-prime) | \$\checkmark\$ | **\$\checkmark\$ (Always Guaranteed!) |**

BCNF

ightarrow Y\$: \$X\$ **MUST** be Super Key | Overlapping candidate keys redundancy | \$\checkmark\$ | \$imes\$ (May be lost) |

4NF	In BCNF + For every \$X\$ woheadrightarrow Y\$: \$X\$ is Super Key	Independent multi-valued attributes (Cartesian product)	\$\checkmark\$	N/A (MVD focus)
5NF (PJNF)	In 4NF + Every \$R_i\$ in JD \$owtie[R_1..R_n]\$ is Super Key	N-way join dependencies & spurious tuples	\$\checkmark\$	N/A

Module 12: AKTU PYQs & Solved Decompositions



> **Course:** Database Management Systems (AKTU Code: **BCS501**)

****Topic Focus:**** Previous Years Exam Questions (2019-2024), 7-Mark & 10-Mark Numerical Solutions, Finding Candidate Keys, Determining Highest Normal Form, Lossless 3NF/BCNF Decomposition.

1. AKTU PYQ 1 (10 Marks — 2022-23 End-Sem)

Question:

Given a relation $R(A, B, C, D, E, F)$ with the following set of functional dependencies:

$$F = \{ A$$

$$\rightarrow B, \quad C$$

$$\rightarrow DE, \quad AC$$

$$\rightarrow F \}$$

- Find all Candidate Keys of R .
- Identify Prime and Non-Prime attributes.
- Determine the highest normal form of R .
- Decompose the relation into 3NF ensuring Lossless Join and Dependency Preservation.

Solution:

Step 1: Candidate Keys

Attributes on RHS: $\{B, D, E, F\}$.

Attributes **NEVER on RHS:** $\{A, C\}$.

Essential attribute set = $\{AC\}$.

Compute closure of $\{AC\}$:

- $\{AC\}^+ = \{A, C\}$

- Using A

$B \rightarrow A \implies \{A, C, B\}$

- Using C

$DE \rightarrow AC \implies \{A, C, B, D, E\}$

- Using AC

$F \rightarrow ABCDE \implies \{A, B, C, D, E, F\} = R$.

Since $\{AC\}^+ = R$ and no proper subset of $\{AC\}$ can determine R , **$\{AC\}$ is the ONLY Candidate Key.**

Step 2: Attribute Classification

Prime Attributes: $\{A, C\}$

Non-Prime Attributes: $\{B, D, E, F\}$

Step 3: Highest Normal Form Test

Let us test each FD in F :

1. A

$B \rightarrow A$:

- Determinant A is a proper subset of Candidate Key $\{AC\}$.

- Dependent B is a **Non-Prime Attribute**.

- This is a **Partial Dependency**!

- **Verdict:** Violates 2NF!

Highest Normal Form = 1NF.

Step 4: Decomposition into 3NF (Using Bernstein Synthesis)

1. Find Canonical Cover F_c :

- $F = \{A$

$B \rightarrow C$

$D \rightarrow C$

$E \rightarrow AC$

$F \rightarrow \}$.

- Check extraneous attributes in AC

$F \rightarrow$:

- $A^+ = \{A, B\}$ (No F).
- $C^+ = \{C, D, E\}$ (No F).
- Neither is extraneous.
- Check redundant FDs: None.
- $F_c = \{A$

$\rightarrow B, \text{quad } C$

$\rightarrow D, \text{quad } C$

$\rightarrow E, \text{quad } AC$

$\rightarrow F \}$.

2. Form Relations from F_c :

- $R_1(\mathbf{A}, B)$ with Key = A (from $A \rightarrow B$).
- $R_2(\mathbf{C}, D, E)$ with Key = C (from $C \rightarrow D, C \rightarrow E$).
- $R_3(\mathbf{A, C}, F)$ with Key = $\{AC\}$ (from $AC \rightarrow F$).

3. Candidate Key Check:

- R_3 already contains the candidate key $\{AC\}$!

4. Final 3NF Relations:

- $\mathbf{R_1(A, B)}$, $\mathbf{R_2(C, D, E)}$, $\mathbf{R_3(A, C, F)}$.
- **Properties:** Lossless Join (✓) and Dependency Preservation (✓).

2. AKTU PYQ 2 (7 Marks — 2021-22 End-Sem)

Question:

Relation $R(A, B, C, D)$ has $F = \{A \rightarrow B$

$\rightarrow C, \text{quad } C$

$\rightarrow D, \text{quad } D$

$\rightarrow A \}$.

Is this relation in 3NF? Is it in BCNF? Justify your answer.

Solution:

1. Candidate Keys:

- $(AB)^+ = \{A, B, C, D\} = R \implies \mathbf{AB}$ is CK.
- $(BC)^+ = \{B, C, D, A\} = R \implies \mathbf{BC}$ is CK.
- $(BD)^+ = \{B, D, A, C\} = R \implies \mathbf{BD}$ is CK.
- Prime Attributes = $\{A, B, C, D\}$. Non-Prime = $\emptyset$.

2. **Test 3NF Condition (For every $X \twoheadrightarrow Y$:

X is Super Key OR Y is Prime):**

- AB

C : AB is Candidate Key (Super Key). $\checkmark$

- C

D : C is not Super Key, but D is **Prime Attribute** (in BD). $\checkmark$

- D

A : D is not Super Key, but A is **Prime Attribute** (in AB). $\checkmark$

- **Relation IS in 3NF!**

3. **Test BCNF Condition (For every $X \twoheadrightarrow Y$:

X MUST be Super Key):**

- In C

D : Determinant C is NOT a Super Key ($C^+ = \{A, C, D\}$ $\neq R$).

- In D

A : Determinant D is NOT a Super Key ($D^+ = \{A, D\}$ $\neq R$).

- **Relation FAILS BCNF!**

4. Conclusion: Relation is in 3NF but NOT in BCNF.

3. AKTU PYQ 3 (10 Marks — 2020-21 End-Sem)

Question:

Explain Lossless Join Decomposition and Dependency Preservation with suitable examples. State the necessary condition for a binary decomposition to be lossless.

Solution:

*(Complete formal definitions, the binary condition theorem $(R_1 \cap R_2)$

$\rightarrow R_1 \cup (R_1 \cap R_2)$

$\rightarrow R_2$, and tabular counter-example demonstrating spurious tuples as detailed in Module 09).*

4. Architectural Diagram
